NAO: 3306

03-10-2025

IN-MEXICO PROGRAM BACKEND DEVELOPER CERTIFICATION

PLAN

Server and Database Commands

Valeria Anel Duque Salgado

Contents:

BACK LOG.

User Story	Requirements
As a developer, I want to create a SerpApi account under the free plan and obtain my API key, so that I can authenticate requests to the Google Scholar API.	○ An active SerpApi account exists under the free plan. ○ The API key is accessible from the SerpApi dashboard. ○ The key is stored securely (environment variable or secrets manager). ○ Documentation explains where to locate the API key.
As a technical writer, I want to prepare a technical report about the Google Scholar API, so that the team has a structured reference guide for using the API.	• The report includes the following sections: ○ Endpoints ○ Authentication methods ○ Query parameters ○ Response formats ○ Usage limits ○ Code examples (at least 2 languages). • The report is saved as a markdown file (technical-report-google-scholar.md). • All sections are clear, accurate, and validated against SerpApi documentation.
As a project maintainer, I want to create a GitHub repository and add initial documentation, so that all team members can collaborate in one central place.	• A new GitHub repository is created with a clear name (e.g., scholar-serpapi-integration). • The repo contains at least: ○ README.md describing project purpose, key functionalities, and relevance. ○ technical-report-google-scholar.md with API documentation. • Repository is initialized with a license and .gitignore.

As a new contributor, I want to understand the project purpose, scope, and setup quickly, so that I can start contributing without confusion.	• README.md includes: • Project purpose (main goal). • Key functionalities (summary of what the project does). • Project relevance (what problem it solves or facilitates). • Setup instructions for using the API key in local development. • Clear links are provided to SerpApi sign-up and API docs. • Example code runs successfully with a valid API key.
As a project reviewer, I want to validate that the technical report and code examples work correctly, so that I can confirm the documentation is accurate and reliable.	• All code examples in the technical report are executed and verified to return results. • The structure of API responses matches the documentation in the report. • A short test script (e.g., test_api_example.js or test_api_example.py) is included in the repository. • The README links to these test files and explains how to run them.

Requirements	Stages	Time estimation	Deliverables
Register for a SerpApi free account. **Confirm email and activate the account.** **Retrieve the personal API key from the dashboard.**	1. Registration: Create the account on SerpApi. 2. Verification: Confirm email and activate access. 3. Configuration: Log in and copy API key for later use.	0.5–1 hr.	Active SerpApi account + working API key stored securely.
Research and document endpoints, authentication, query parameters, response formats, usage limits, and code examples.	1. Research: Gather official SerpApi/Google Scholar API documentation and examples. 2. Structuring: Organize findings according to the required outline (endpoints, authentication, etc.). 3. Drafting: Write detailed explanations with examples. 4. Review: Check clarity, accuracy, and consistency.	4–5 hrs.	Completed Technical Report (Markdown or PDF) covering all requested sections.
Create repository. **Add README with purpose, functionalities, and relevance.** **Upload Technical Report.**	1. Repo Creation: Open GitHub, create new repository with suitable name. 2. Initial Commit: Add basic README.md with project overview. 3. Documentation Upload: Place the	3 hrs.	Test scripts + validation notes proving accuracy of the documentation.

Configure access permissions for Digital NAO team.	Technical Report inside the repo. 4. Permissions Setup: Invite Digital NAO team members and assign proper access roles.		
Verify code examples, create test script, and document results.	1. Execution: Run provided code examples (Python, JS, etc.) against the API. 2. Validation: Compare API responses to what is described in the Technical Report. 3. Test Script Creation: Write one or more small scripts to confirm reproducibility. 4. Documentation: Add notes in the repo (README or separate doc) on how to execute tests..	2–3 days.	Integration algorithm. Consolidated CSV output. End-to-end test report.

Update README with setup instructions, usage steps, and troubleshooting. **Ensure clear structure in the repo.**	1. User Guidance Draft: Write instructions for installing dependencies, configuring API key, and running examples. 2. File Organization: Arrange repository files logically (e.g., /docs, /examples, /tests). 3. Troubleshooting Section: Add FAQ or common issues with solutions.	1.5–2 hrs.	Enhanced README with onboarding guide + organized file structure.